KINESICS FLASH ON SYSTEM

A
Major Project Stage-I Report

Submitted in partial fulfillment of
the requirements for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering

Submitted by

A. Prathibha	**(20SS1A0543)**
G. Kalyan	**(20SS1A0520)**
O.V.V.S Ganapathi	**(20SS1A0536)**
S. Sai Charan	**(20SS1A0549)**
V. Gayathri	**(21SS5A0514)**

Under the guidance of

Mrs. K.Neeraja
Assistant Professor(C)

Department of Computer Science and Engineering

JNTUH University College of Engineering Sultanpur

Sultanpur(V),Pulkal(M),Sangareddy (Dist.),Telangana-502273

December 2023

JNTUH UNIVERSITY COLLEGE OF ENGINEERING SULTANPUR

Sultanpur(V),Pulkal(M),Sangareddy(Dist.),Telangana-502273

Department of Computer Science and Engineering

Certificate

This is to certify that the Major Project report work entitled **"KINESICS FLASH ON SYSTEM "** is a bonafide work carried out by a team consisting of **A. Prathibha** bearing Roll no.**20SS1A0543**, **G. Kalyan** bearing Roll no.**20SS1A0520**, **O.V.V.S Ganapathi** bearing Roll no.**20SS1A0536**, **S. Sai Charan** bearing Roll no.**20SS1A0549**, **V. Gayathri** bearing Roll no.**21SS5A0514** in partial fulfillment of the requirements for the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING discipline to Jawaharlal Nehru Technological University Hyderabad University College of Engineering Sultanpur during the academic year 2023- 2024.

The results embodied in this report have not been submitted to any other University or Institution for the award of any degree.

Guide
Mrs. K.Neeraja
Assistant Professor(C)

Head of the Department
Dr. B.V. Ram Naresh Yadav
Professor

External Examiner

Declaration

We hereby declare that Major Project Stage-I entitled "**KINESICS FLASH ON SYSTEM** " is a bonafide work carried out by a team consisting of **A. Prathibha** bearing Roll no.**20SS1A0543**, **G. Kalyan** bearing Roll no.**20SS1A0520**, **O.V.V.S Ganapathi** bearing Roll no.**20SS1A0536**, **S. Sai Charan** bearing Roll no. **20SS1A0549**, **V. Gayathri** bearing Roll no. **21SS5A0514** in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering discipline to Jawaharlal Nehru Technological University Hyderabad University College of Engineering Sultanpur during the academic year 2023- 2024.The results emboided in this report have not been submitted to any other University or Institution for the award of any degree.

A. Prathibha **(20SS1A0543)**

G. Kalyan **(20SS1A0520)**

O.V.V.S Ganapathi **(20SS1A0536)**

S. Sai Charan **(20SS1A0549)**

V. Gayathri **(21SS5A0514)**

Acknowledgement

We wish to take this opportunity to express our deep gratitude to all those who helped us in various ways during our Major Project report work. It is our pleasure to acknowledge the help of all those individuals who were responsible for foreseeing the successful completion of our Major Project report.

We express our sincere gratitude to **Dr. G. Narsimha, Professor and Principal**, JNTUHUCES for his support during the course period.

We express our sincere gratitude to **Dr. Y. Raghavender Rao, Professor, Vice Principal and Head of the Department(ECE)**, JNTUHUCES for his effective suggestions during the course period.

We are thankful to **Dr. B. V. Ram Naresh Yadav, Professor, and Head of the Department**, Computer Science and Engineering, for his support and guidance in the completion of our Major Project.

We are thankful to our Guide **Mrs. K. Neeraja, Assistant Professor(C)** for her support and encouragement throughout the course period.

Finally,we express our gratitude with great admiration and respect to our faculty for their moral support and encouragement throughout the course.

A. Prathibha (20SS1A0543)
G. Kalyan (20SS1A0520)
O.V.V.S Ganapathi (20SS1A0536)
S. Sai Charan (20SS1A0549)
V. Gayathri (21SS5A0514)

Contents

Abstract

The Kinesics Flash On system is an advanced gesture recognition platform that seamlessly translates facial and hand actions into text while simultaneously converting facial expressions into corresponding emojis. This project attempts to bridge communication gaps between the deaf and hearing communities by developing an innovative AI-powered solution. This system utilizes computer vision and machine learning techniques to accurately recognize and translate finger spelling kinesics into text language, enabling seamless communication between individuals who use finger spelling and those who do not. The project consists of several key components, including data collection, model training, kinesics recognition, and translation. The system's machine learning component is then harnessed to convert the recognized kinesics into corresponding textual representations. To make this work, we use special computer programs that recognize different hand movements and patterns. These programs learn from a lot of examples to understand what each hand kinesics means. When someone finger spells, the system looks at their hand movements through a camera and quickly changes them into words on a screen. This way, people who use finger spelling can talk to anyone, even if the other person does not know finger spelling. The system gets better over time because it keeps learning from people's finger spelling and improving its translations. This project is a way to help everyone communicate better and make sure nobody feels left out when they want to talk.

List of Figures

Chapter 1

INTRODUCTION

The Kinesics Flash On system represents a groundbreaking advancement in communication technology. By harnessing the power of computer vision and machine learning, this innovative platform seamlessly translates facial expressions and hand gestures, particularly finger spelling kinesics, into both text and corresponding emojis. Through a process of data collection, model training, and real-time recognition, the system enables individuals who use finger spelling to communicate effortlessly with those who do not. The machine learning component continuously refines its understanding by learning from numerous examples, ensuring accurate and evolving translations over time. Ultimately, this project is a transformative step towards inclusive communication, ensuring that everyone can engage in meaningful conversations, regardless of their familiarity with finger spelling.

1.1 Project Overview

The Gesture Recognition System is a pioneering technology designed to empower and inclusively connect individuals, particularly benefiting the deaf and mute community. Utilizing advanced computer vision and convolutional neural network (CNN) models, the system captures real-time hand and facial gestures through video streaming. Its primary goal is to bridge communication gaps, translating hand gestures into text and facial expressions into emojis. With a focus on efficiency and versatility, the system employs Tkinter for real-time image processing, recognizing specific actions like greetings or

approvals. Overall, this innovative solution serves as a transformative tool for enhancing communication and interaction, making it more intuitive and meaningful.

Individuals who are deaf and mute encounter significant communication barriers that necessitate the development of advanced solutions. Current gesture recognition systems face challenges, particularly in accurately interpreting nuanced facial expressions, hindering their effectiveness. The limitations extend to issues related to real-time image processing and responsiveness, especially with the introduction of convolutional neural network (CNN) models, which add complexity to system efficiency and adaptability. Recognizing specific actions beyond basic gestures is another notable challenge. The practical implementation of such systems often struggles to provide seamless user experiences. Addressing these limitations is crucial in creating a comprehensive solution that accurately translates real-time gestures into text and emojis.

1.2 Problem Statement

Individuals who are deaf and mute encounter significant communication barriers that necessitate the development of advanced solutions. Current gesture recognition systems face challenges, particularly in accurately interpreting nuanced facial expressions, hindering their effectiveness. The limitations extend to issues related to real-time image processing and responsiveness, especially with the introduction of convolutional neural network (CNN) models, which add complexity to system efficiency and adaptability. Recognizing specific actions beyond basic gestures is another notable challenge. The practical implementation of such systems often struggles to provide seamless user experiences. Addressing these limitations is crucial in creating a comprehensive solution that accurately translates real-time gestures into text and emojis. This solution must not only cater to diverse user needs and scenarios but also overcome challenges related to accuracy, adaptability, and user experience. Therefore, the development of an advanced Gesture Recognition System is imperative, not only to empower the deaf and mute community but also to enhance human-computer interaction on a broader scale.

1.3 Purpose

The Gesture Recognition System empowers the deaf and mute community by enabling communication through intuitive gestures. It enhances human-computer interaction by capturing real-time hand and facial gestures, bridging communication gaps and conveying emotions seamlessly. Incorporating advanced technologies like convolutional neural network (CNN) models ensures high accuracy, transforming interaction into a more inclusive and accessible experience.Individuals who are deaf and mute encounter significant communication barriers that necessitate the development of advanced solutions. Current gesture recognition systems face challenges, particularly in accurately interpreting nuanced facial expressions, hindering their effectiveness. The limitations extend to issues related to real-time image processing and responsiveness, especially with the introduction of convolutional neural network (CNN) models, which add complexity to system efficiency and adaptability. Recognizing specific actions beyond basic gestures is another notable challenge. The practical implementation of such systems often struggles to provide seamless user experiences. Addressing these limitations is crucial in creating a comprehensive solution that accurately translates real-time gestures into text and emojis. This solution must not only cater to diverse user needs and scenarios but also overcome challenges related to accuracy, adaptability, and user experience. Therefore, the development of an advanced Gesture Recognition System is imperative, not only to empower the deaf and mute community but also to enhance human-computer interaction on a broader scale.

1.4 Existing System

1.4.1 Electronic-Based Gesture Recognition System

An electronic-based gesture recognition system is a technology that enables devices to interpret and respond to human gestures. Instead of using traditional input devices like keyboards or mice, these systems allow users to interact with devices using hand movements, body gestures, or facial expressions. The primary goal is to create a more intuitive and natural user interface.

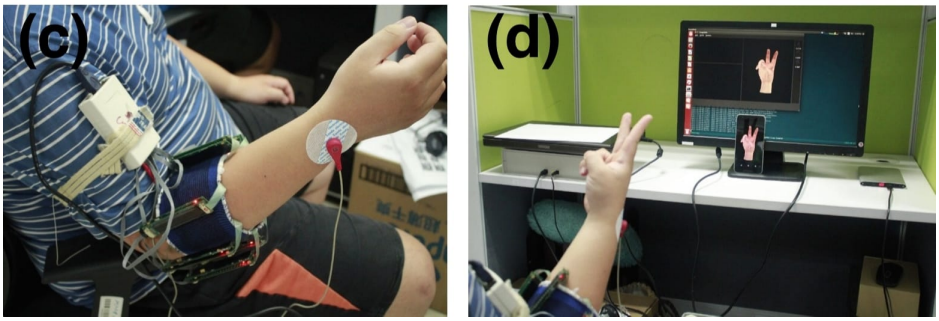

Figure 1.1: Electronic Based

1.4.2 Glove-Based Gesture Recognition System

A glove-based gesture recognition system involves the use of a specially designed glove equipped with sensors and technology to detect and interpret hand and finger movements. This system enables users to interact with devices or computer systems by making gestures with their hands while wearing the glove. The primary objective is to provide a natural and intuitive means of communication, particularly in applications like virtual reality, gaming, robotics, and human-computer interaction.

Figure 1.2: Glove Based System

1.4.3 Marker-Based Gesture Recognition System

A marker-based gesture recognition system involves the use of physical markers, typically visual cues or patterns, to track and interpret human gestures. These markers serve as reference points for the system to identify and analyze specific movements or poses.

Marker-based systems are commonly employed in applications such as motion capture, augmented reality, and virtual reality.

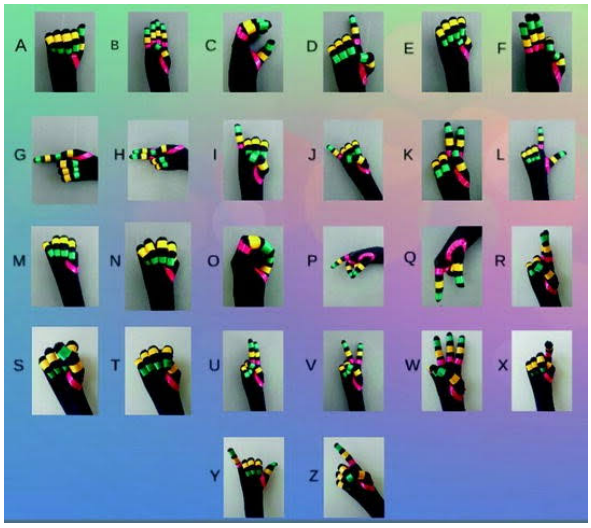

Figure 1.3: Marker Based System

1.5 Proposed System

The proposed gesture recognition system integrates cutting-edge Computer Vision and Machine Learning techniques, primarily leveraging Convolutional Neural Networks (CNN). The system is designed to capture and interpret hand and face gestures in real-time through a camera. The CNN model, employed for both hand and face gestures, plays a pivotal role in achieving high accuracy. The hand gesture recognition module translates detected hand movements into corresponding text, enhancing the system's communicative capabilities. Simultaneously, the face gesture recognition module converts facial expressions into emojis, providing a visually intuitive aspect to the system. To ensure accuracy, the system undergoes rigorous training using a dataset and algorithms, and its performance is validated through testing with captured images in real-time. The emphasis on a vector-based representation enhances the system's efficiency and adaptability, enabling standardized encoding of gestures. Additionally, the system is geared towards implementing an extensive repertoire of gestures, fostering a more nuanced and diverse interaction. The combination of both hand and face gestures enriches the user experience, allowing for a comprehensive and expressive form of communication. Through these innovations, the proposed system aims to set a benchmark in accuracy, versatility, and user engagement in the domain of gesture recognition.

1.5.1 Hand Gesture Recognition

Hand gesture recognition is a technology that employs computer vision and machine learning to interpret and respond to hand movements, enabling human-computer interaction. The process consists of data collection, where images or videos of various hand gestures are gathered for a dataset. This data undergoes preprocessing to enhance quality, including resizing and noise reduction. Essential features, such as hand shape and finger positions, are extracted.

Machine learning models, often based on Convolutional Neural Networks (CNNs), are then trained using the preprocessed data. These models learn to associate specific features with corresponding gestures. During real-time operation, the system captures video frames, processes them, and classifies hand gestures using the trained model. The recognized gestures can be translated into commands for computer interaction. Key steps include continuous data collection, feature extraction, model training, real-time detection, and interaction feedback. Advances in deep learning, particularly CNNs, have significantly improved the accuracy and efficiency of hand gesture recognition systems.

1.5.2 Face Gesture Recognition

The face to emoji recognition process begins by isolating facial features in an input image. These features are used to create a detailed 'Face Vector' and 'Expressional Vector,' which, along with a Convolutional Neural Network, enable accurate mood classification and subsequent translation of predicted emotions into corresponding emojis. This comprehensive pipeline allows for precise interpretation and visual representation of the individual's emotional state through expressive emojis.

1.6 Scope

This paper viewed the existing system and the pros of it. There are different systems for gesture recognition system but each system contains a disadvantage that makes it not to use. We are here to propose kinesics flash on system using python tools which helps in the gesture recognition with less effort and more accuracy than the existing system.

1.7 Conclusion

In conclusion, the proposed Kinesics Flash On system stands as a pioneering advancement in communication technology, specifically designed to empower and connect the deaf and mute community. By seamlessly translating facial expressions and finger spelling kinesics into text and emojis through computer vision and machine learning, the system aims to bridge communication gaps. Addressing limitations of existing gesture recognition systems, the project emphasizes high accuracy, real-time processing, and an extensive repertoire of gestures. With a focus on inclusivity, efficiency, and user engagement, the Kinesics Flash On system strives to set a benchmark in accuracy and versatility, offering a transformative tool for intuitive and meaningful communication.

Chapter 2

LITERATURE SURVEY

2.1 Introduction

Hasan [17] applied multivariate Gaussian distribution to recognize hand gestures using non-geometric features. The input hand image is segmented using two different methods [18]; skin color based segmentation by applying HSV color model and clustering based thresholding techniques [18]. Some operations are performed to capture the shape of the hand to extract hand feature; the modified Direction Analysis Algorithm are adopted to find a relationship between statistical parameters (variance and covariance) [17] from the data, and used to compute object(hand) slope and trend [17] by finding the direction of the hand gesture [17], As shown in Figure

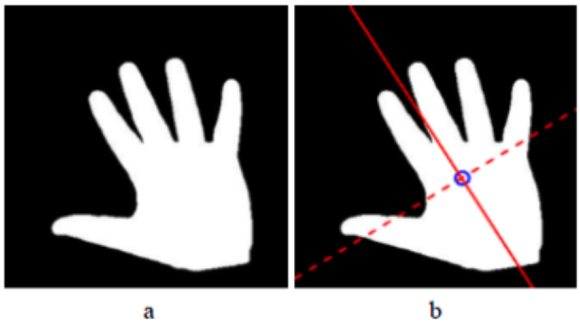

Figure 2.1: Computing Hand Direction [17]

Then Gaussian distinction is applied on the segmented image, and it takes the direction of the hand as shown in figure.

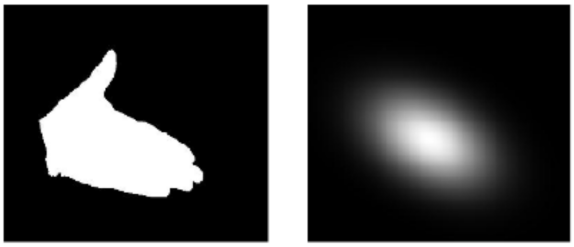

Figure 2.2: Gaussian Distribution Applied on the Segmented Image [18]

Form the resultant Gaussian function the image has been divided into circular regions in other words that regions are formed in a terrace shape so that to eliminate the rotation affect [17][18].The shape is divided into 11 terraces with a 0.1 width for each terrace [17][18]. 9 terraces are resultant from the 0.1 width division which are; (1-0.9, 0.9-0.8, 0.8-0.7, 0.7-0.6, 0.6, 0.5, 0.5-0.4,0.4-0.3, 0.3-0.2, 0.2-0.1), and one terrace for the terrace that has value smaller than 0.1 and the last one for the external area that extended out of the outer terrace [17][18]. An explanation of this division is demonstrated in Figure .

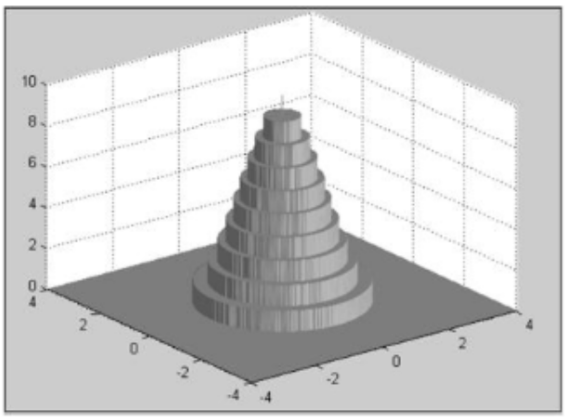

Figure 2.3: Terraces Division with 0.1 Likelihood [17][18]

Each terrace is divided into 8 sectors which named as the feature areas, empirically discovered that number 8 is suitable for features divisions [17], To attain best capturing of the Gaussian to fit the segmented hand, re-estimation are performed on the shape to fit capturing the hand object[17], then the Gaussian shape are matched on the segmented hand to prepare the final hand shape for extracting the features, Figure 8 shown this process [17]. International Journal of Artificial Intelligence Applications (IJAIA), Vol.3, No.4, July 2012

Kulkarni [31] recognize static posture of American Sign Language using neural networks algorithm. The input image are converted into HSV color model, resized into 80x64 and some image preprocessing operations are applied to segment the hand [31]from a uniform background[31], features are extracted using histogram technique and Hough algorithm. Feed forward Neural Networks with three layers are used for gesture

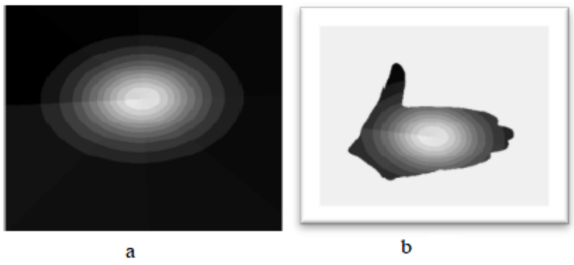

Figure 2.4: Features Divisions [18]. a) Terrace Area in Gaussian. b) Terrace area in Hand Image

classification. 8 samples are used for each 26characters in sign language, for each gesture, 5 samples are used for training and 3samples fortesting, the system achieved 92.78International Journal of Artificial Intelligence Applications (IJAIA), Vol.3, No.4, July 2012169and 5 samples for testing. The recognition rate for the normalized feature problem achieved better performance than the normal feature method, 95

Figure 2.5: Applying Trimming Process on the Input Image, Followed By Scaling Normalization Process [5]

Wysoski et al. [8] presented rotation invariant postures using boundary histogram. Camera used for acquire the input image, filter for skin color detection has been used followed by clustering process to find the boundary for each group in the clustered image using ordinary contour-tracking algorithm. The image was divided into grids and the boundaries have been normalized.The boundary was represented as chord's size chain which has been used as histograms, by dividing the image into number of regions N in a radial form, according to specific angle. For classification process Neural Networks MLP and Dynamic Programming DP matching were used. Many experiments have implemented on different features format in addition to used ifferent chord's size histogram, chord's size FFT. 26 static postures from American Sign Language used in the experiments. Homogeneous background was applied in the work.Stergiopoulou [14] suggested a new Self-Growing and Self-Organized Neural Gas (SGONG)network for hand gesture recognition. For hand region detection a color segmentation technique based on skin color filter in the YCbCr color space was used, an approximation of hand shape morphology has been detected using (SGONG) network; Three features were extracted using finger identification process which determines the number of the raised fingers and characteristics

of hand shape.

2.2 Conclusion

In conclusion, the literature survey on gesture recognition focusing on hand and face has provided valuable insights into the advancements, challenges, and potential applications of this rapidly evolving field. The comprehensive review of existing research has highlighted the diverse range of techniques and methodologies employed for recognizing gestures, particularly those performed by the hand and face.

Chapter 3

REQUIREMENT SPECIFICATION

3.1 Software Requirements

Software Requirements for the project are:

- Programming Language-Python
- Computer Vision Libraries-OpenCV
- Machine Learning Frameworks.
- Text Translation APIs
- Integrated Development Environment (IDE)

3.2 Hardware Requirements

Hardware Requirements for the project are:

- I5 processor or higher
- Hard Disk minimum of 40 GB

- RAM minimum of 2GB
- Integrated webcam or external webcam(15-20fps)

3.3 Conclusion

Hardware and Software are required to make a computer system to operate effectively. Hardware is the physical components while the software forms the interface between the hardware and software.

Chapter 4

WORKING OF SYSTEM

4.1 System Architecture

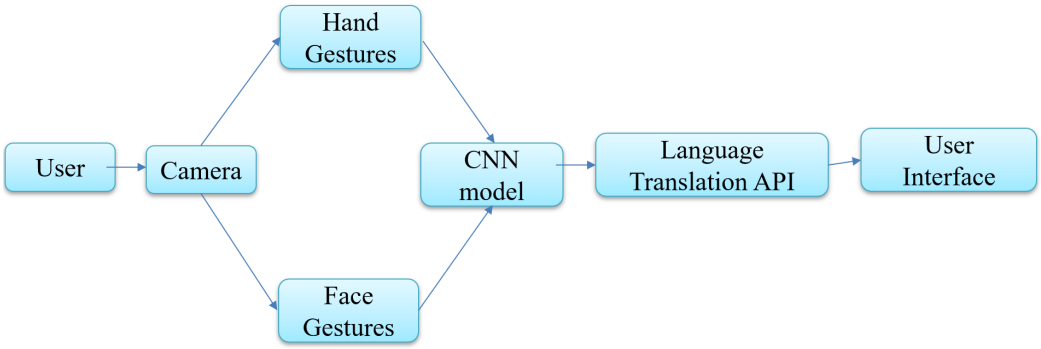

Figure 4.1: Architecture

The image presents a detailed flowchart demonstrating the architecture of a language translation system focused on understanding and converting hand gestures into spoken words. The system in question consists of four main elements being at camera, a Convolutional Neural Network (CNN) model, a Language Translation API, and a user interface. All these components are labelled in the flowchart and they imply an operation where the user's hand gestures are captured by the camera, analysed by the neural network (CNN Model), translated into spoken words by the Language Translation API, and ultimately presented to the user via some user interface. The general theme of the image is technical, with a strong focus on explaining this specific architectural design of the system.

4.2 Machine Learning

Kinesics flash on system relies on machine learning technology, which requires massive data sets to learn to deliver accurate results.

4.2.1 Hand Gesture Recognition

Hand gesture recognition involves the utilization of computer vision and machine learning techniques to interpret and understand the movements and positions of the hand, enabling human-computer interaction.

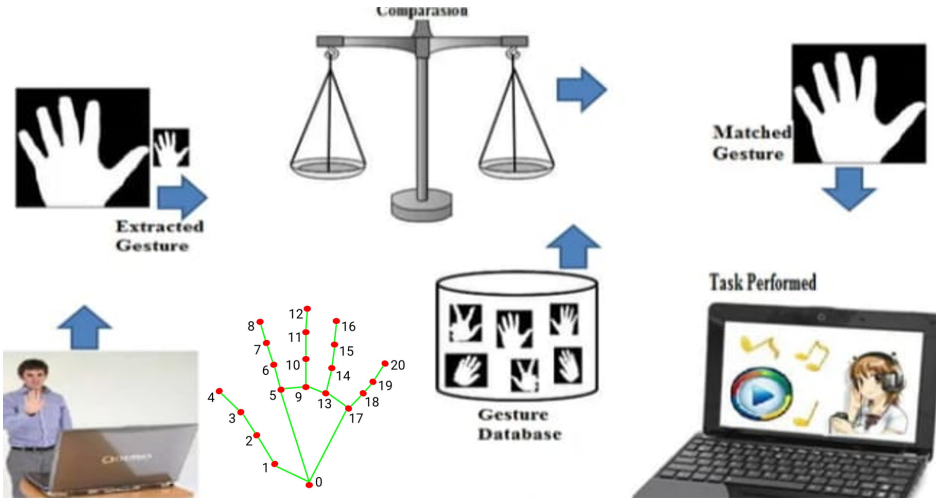

Figure 4.2: Hand Gesture Recognition Flow

The process can be broken down into several key steps:

- Data Collection: Hand gesture recognition systems begin by collecting a dataset of images or videos featuring various hand gestures. This dataset is crucial for training the machine learning model.
- Image Preprocessing: The collected data undergoes preprocessing to enhance its quality and extract relevant features. Techniques such as resizing, normalization, and noise reduction are applied to ensure uniformity and clarity.
- Feature Extraction: Extracting essential features from the preprocessed images is a critical step. Features may include the shape of the hand, finger positions, and relative

distances between fingers. These features form the basis for the model to learn and recognize different gestures.

- Model Training: Machine learning models, often based on deep learning architectures like Convolutional Neural Networks (CNN), are trained using the preprocessed and feature-extracted data. The model learns to associate specific features with corresponding hand gestures.
- Gesture Classification: Once the model is trained, it can classify new hand gestures based on the features it has learned during training. During real-time operation, the system captures images of hand gestures and feeds them into the trained model for classification
- Real-Time Detection: The system continuously captures video frames in real-time through a camera. These frames are then processed, and the hand gestures are determined by the trained model. The system interprets the positions, movements, and configurations of the fingers and palm to identify the specific gesture being performed.

4.2.2 Face Gesture Recognition Flow

Face gestures recognition to emoji conversion involves a series of modules that leverage computer vision and machine learning for accurate detection and interpretation of facial expressions

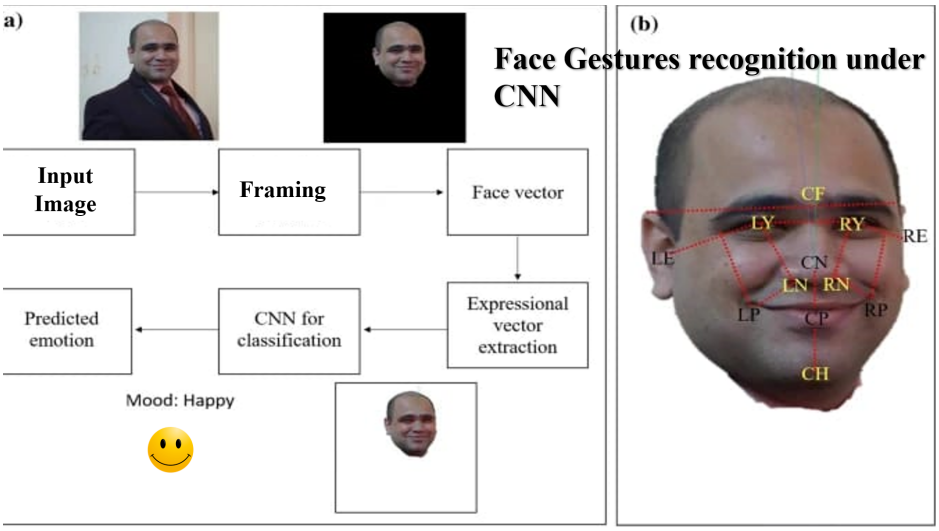

Figure 4.3: Face Gesture Recognition Flow

- Face Detection Module: This initial module identifies and locates faces within images or video streams, laying the groundwork for subsequent facial analysis.
- Facial Landmark Detection Module: This module precisely identifies key facial landmarks such as eyes, eyebrows, nose, and mouth. It captures subtle movements and configurations critical for understanding facial expressions.
- Expression Analysis Module: Employing computer vision algorithms and often deep learning models, this module analyzes the movements of facial features to recognize and classify different expressions, such as happiness, sadness, surprise, etc.
- Expression-to-Emoji Mapping Module: Recognized facial expressions are mapped to corresponding emojis through predefined associations. Each emoji symbolizes a specific emotional state, creating a direct link between the user's expression and the digital representation.
- Real-Time Processing Module: Operating in real-time, this module continuously captures and processes video frames, ensuring instantaneous feedback and responsiveness to the user's changing facial expressions.

4.2.3 Text Translation

Text translation from hand gestures involves real-time identification and tracking of hand movements through computer vision. Extracted features such as finger positions are then classified using machine learning models, facilitating instantaneous conversion of gestures into textual information. On the other hand, face gestures are translated into emojis through the recognition of facial expressions using computer vision. Predefined mappings associate facial expressions with specific emojis, enabling dynamic and real-time feedback. Both technologies enrich human-computer interaction by translating non-verbal cues into meaningful digital outputs, whether as text or emojis.

Chapter 5

SYSTEM DESIGN

5.1 UML Diagrams

UML, which stands for Unified Modeling Language, is a way to visually represent the architecture, design, and implementation of complex software systems UML is a standardized modeling language that can be used across different programming languages and development processes, so the majority of software developers will understand it and be able to apply it to their work.

5.1.1 Use Case Diagram

A use case diagram is used to represent the dynamic behavior of a system. It models the tasks, services, and functions required by a system /subsystem of an application. It depicts the high-level functionality of a system and also tells how the user handles

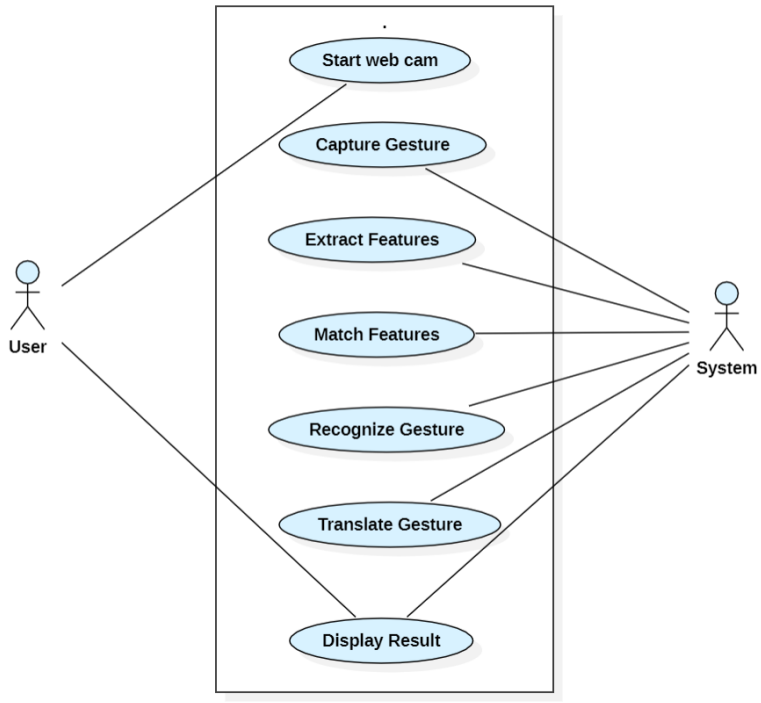

Figure 5.1: Use Case Diagram

5.1.2 Class Diagram

The diagram depicts a static view of an application. It represents the types of objects residing in the system and the relationships between them. A class consists of its objects, and also it may inherit from other classes. A class diagram is used to visualize, describe, document various different aspects of the system, and also construct executable software code.

5.1.3 Sequence Diagram

Sequence Diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration. Sequence Diagrams are time focus and they show the order of the interaction visually by using the vertical axis of the diagram to represent time what messages are sent and when.

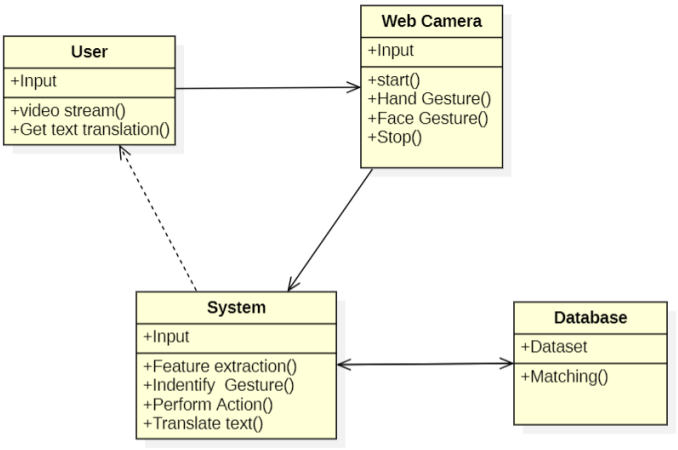

Figure 5.2: Class Diagram

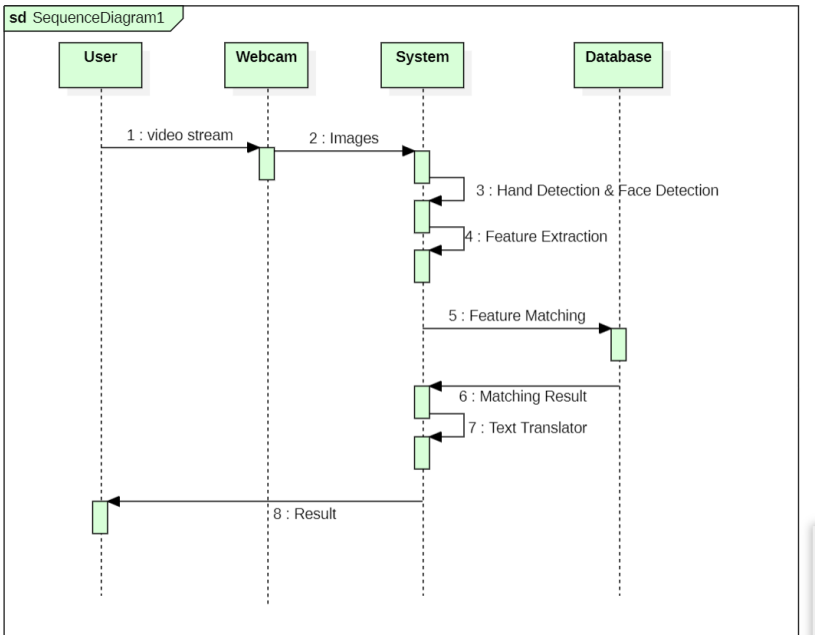

Figure 5.3: Sequence Diagram

5.1.4 Communication Diagram

UML Communication Diagrams, previously known as collaboration diagrams are a type of behavioural diagram that shows the interactions that take place between objects in a piece of software or system. This type of diagram emphasizes the messages exchanged between

objects. Communication diagrams are best used when one use case has multiple scenarios that need depicting.

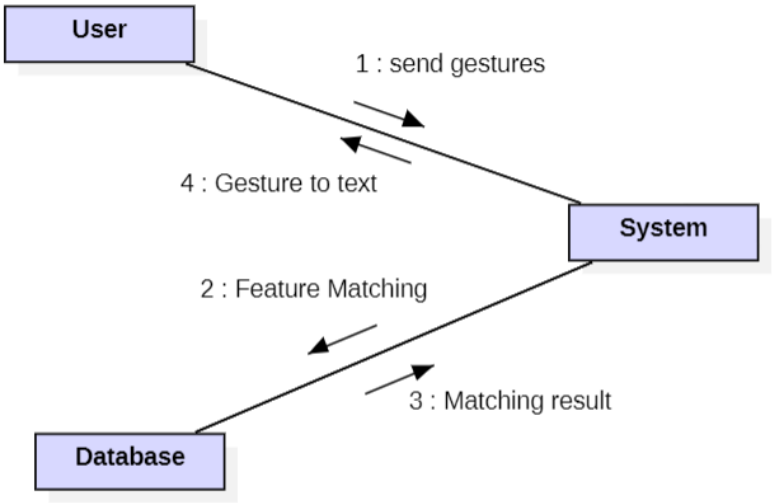

Figure 5.4: Communication Diagram

5.1.5 Activity Diagram

The Activity diagram is used to demonstrate the flow of control within the system rather than the implementation. The activity diagram helps in envisioning the workflow from one activity to another. It puts emphasis on the condition of flow and the order in which it occurs.

5.1.6 State Diagram

A state diagram is a type of diagram used in computer science and related fields to describe the behavior of systems. State diagrams require that the system described is composed of a finite number of states; sometimes, this is indeed the case, while at other times this is a reasonable abstraction. Many forms of state diagrams exist, which differ slightly and have different semantics.

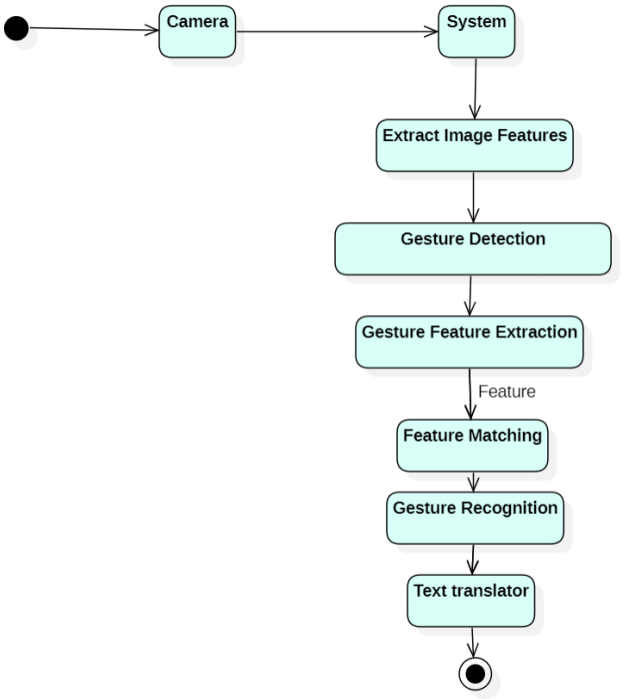

Figure 5.5: Activity Diagram

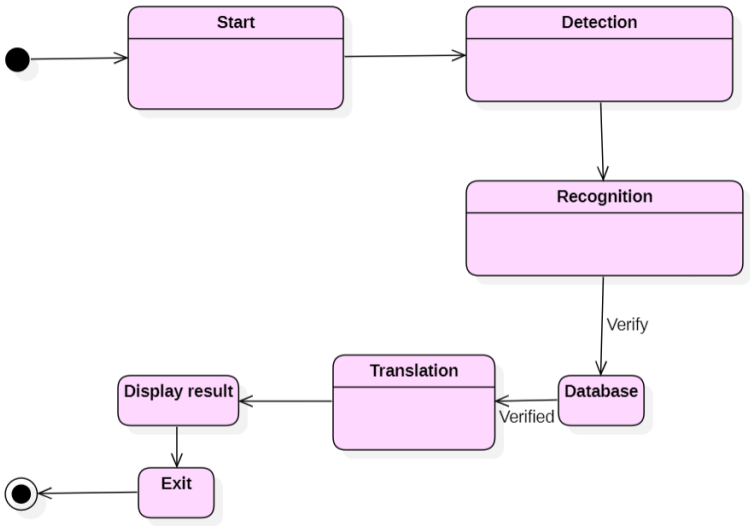

Figure 5.6: State Diagram

5.2 Conclusion

The prevailing methodology used to specify software designs of UML. This language consists of various types of diagrams, each one dedicated to a different design aspect. This variety of views, that overlap with respect to the information depicted in each can leave the overall system design specification in an inconsistent state

report graphicx

listings xcolor

report graphicx

listings xcolor

Chapter 6

Implementation

6.1 HandGesture Recognition

6.1.1 Training

```
1
2     import numpy as np
3 import cv2
4 from tensorflow.keras.models import Sequential
5 from keras.layers import Dense, Dropout, Flatten
6 from keras.layers import Conv2D
7 from keras.optimizers import Adam
8 from keras.layers import MaxPooling2D
9 from keras.preprocessing.image import ImageDataGenerator
10
11 train_dir = 'data/train'
12 val_dir = 'data/test'
13 train_datagen = ImageDataGenerator (rescale=1./255)
14 val_datagen =ImageDataGenerator (rescale=1./255)
15 train_generator= train_datagen.flow_from_directory(
16         train_dir,
17         target_size=(48,48),
18         batch_size=64,
19         color_mode="grayscale",
20         class_mode='categorical')
21 validation_generator = val_datagen.flow_from_directory(
22         val_dir,
```

```
        target_size=(48,48),
        batch_size=64,
        color_mode="grayscale",
        class_mode='categorical')


emotion_model = Sequential()
emotion_model.add(Conv2D(32, kernel_size=(3, 3), activation='
    relu', input_shape=(48,48,1)))
emotion_model.add(Conv2D(64, kernel_size=(3, 3), activation='
    relu'))
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
emotion_model.add(Dropout(0.25))
emotion_model.add(Conv2D(128, kernel_size=(3, 3), activation='
    relu'))
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
emotion_model.add(Conv2D(128, kernel_size=(3, 3), activation='
    relu'))
emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
emotion_model.add(Dropout(0.25))
emotion_model.add(Flatten())
emotion_model.add(Dense (1024, activation ='relu'))
emotion_model.add(Dropout(0.5))
emotion_model.add(Dense (7, activation='softmax'))

learning_rate = 0.0001
decay_rate = 1e-6
optimizer = Adam(learning_rate=learning_rate)

emotion_model.compile(loss="categorical_crossentropy",
    optimizer=optimizer, metrics=['accuracy'])

emotion_model_info = emotion_model.fit_generator(
        train_generator,
        steps_per_epoch=28709 // 64,
        epochs=50, # 50
        validation_data=validation_generator,
        validation_steps=7178 // 64)

emotion_model.save_weights('model.h5')
```

6.1.2 Prediction

```

```

```
from tkinter import messagebox
from tkinter import *
from tkinter import simpledialog
import tkinter
from tkinter import filedialog
from tkinter.filedialog import askopenfilename
import cv2
import random
import numpy as np
from keras.utils import to_categorical
from keras.layers import  MaxPooling2D
from keras.layers import Dense, Dropout, Activation, Flatten
from keras.layers import Convolution2D
from keras.models import Sequential
from keras.models import model_from_json
import pickle
import os
import imutils
from gtts import gTTS
from playsound import playsound
import os
from threading import Thread
from googletrans import Translator

main = tkinter.Tk()
main.title("Sign Language Recognition to Text & Voice using CNN
    Advance")
main.geometry("1300x1200")

global filename
global classifier

bg = None
playcount = 0

#names = ['Palm','I','Fist','Fist Moved','Thumbs up','Index','
   OK','Palm Moved','C','Down']
names = ['C','Thumbs Down','Fist','I','Ok','Palm','Thumbs up']

def getID(name):
    index = 0
    for i in range(len(names)):
        if names[i] == name:
            index = i
            break
    return index
```

```
46
47
48 bgModel = cv2.createBackgroundSubtractorMOG2(0, 50)
49
50 def deleteDirectory():
51     filelist = [ f for f in os.listdir('play') if f.endswith(".
       mp3") ]
52     for f in filelist:
53         os.remove(os.path.join('play', f))
54
55 def play(playcount,gesture):
56     class PlayThread(Thread):
57         def __init__(self,playcount,gesture):
58             Thread.__init__(self)
59             self.gesture = gesture
60             self.playcount = playcount
61
62         def run(self):
63             t1 = gTTS(text=self.gesture, lang='en', slow=False)
64             t1.save("play/"+str(self.playcount)+".mp3")
65             playsound("play/"+str(self.playcount)+".mp3")
66
67
68     newthread = PlayThread(playcount,gesture)
69     newthread.start()
70
71 def remove_background(frame):
72     fgmask = bgModel.apply(frame, learningRate=0)
73     kernel = np.ones((3, 3), np.uint8)
74     fgmask = cv2.erode(fgmask, kernel, iterations=1)
75     res = cv2.bitwise_and(frame, frame, mask=fgmask)
76     return res
77
78 def uploadDataset():
79     global filename
80     global labels
81     labels = []
82     filename = filedialog.askdirectory(initialdir=".")
83     pathlabel.config(text=filename)
84     text.delete('1.0', END)
85     text.insert(END,filename+" loaded\n\n");
86
87
88
89 def trainCNN():
90     global classifier
```

```
    text.delete('1.0', END)
    X_train = np.load('model1/X.txt.npy')
    Y_train = np.load('model1/Y.txt.npy')
    text.insert(END,"CNN is training on total images : "+str(
len(X_train))+"\n")

    if os.path.exists('model1/model.json'):
        with open('model1/model.json', "r") as json_file:
            loaded_model_json = json_file.read()
            classifier = model_from_json(loaded_model_json)
        classifier.load_weights("model1/model_weights.h5")
        classifier._make_predict_function()
        print(classifier.summary())
        f = open('model1/history.pckl', 'rb')
        data = pickle.load(f)
        f.close()
        acc = data['accuracy']
        accuracy = acc[9] * 100
        text.insert(END,"CNN Hand Gesture Training Model
Prediction Accuracy = "+str(accuracy))
    else:
        classifier = Sequential()
        classifier.add(Convolution2D(32, 3, 3, input_shape =
(64, 64, 3), activation = 'relu'))
        classifier.add(MaxPooling2D(pool_size = (2, 2)))
        classifier.add(Convolution2D(32, 3, 3, activation = '
relu'))
        classifier.add(MaxPooling2D(pool_size = (2, 2)))
        classifier.add(Flatten())
        classifier.add(Dense(output_dim = 256, activation = '
relu'))
        classifier.add(Dense(output_dim = 5, activation = '
softmax'))
        print(classifier.summary())
        classifier.compile(optimizer = 'adam', loss = '
categorical_crossentropy', metrics = ['accuracy'])
        hist = classifier.fit(X_train, Y_train, batch_size=16,
epochs=10, shuffle=True, verbose=2)
        classifier.save_weights('model1/model_weights.h5')
        model_json = classifier.to_json()
        with open("model1/model.json", "w") as json_file:
            json_file.write(model_json)
        f = open('model1/history.pckl', 'wb')
        pickle.dump(hist.history, f)
        f.close()
        f = open('model1/history.pckl', 'rb')
```

```
        data = pickle.load(f)
        f.close()
        acc = data['accuracy']
        accuracy = acc[9] * 100
        text.insert(END,"CNN Hand Gesture Training Model
    Prediction Accuracy = "+str(accuracy))



def run_avg(image, aWeight):
    global bg
    if bg is None:
        bg = image.copy().astype("float")
        return
    cv2.accumulateWeighted(image, bg, aWeight)

def segment(image, threshold=25):
    global bg
    diff = cv2.absdiff(bg.astype("uint8"), image)
    thresholded = cv2.threshold(diff, threshold, 255, cv2.
    THRESH_BINARY)[1]
    ( cnts, _) = cv2.findContours(thresholded.copy(), cv2.
    RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if len(cnts) == 0:
        return
    else:
        segmented = max(cnts, key=cv2.contourArea)
        return (thresholded, segmented)

def translate(text, target_language):
    translator = Translator()
    translated_text = translator.translate(text, dest=
    target_language)
    return translated_text.text

def webcamPredict():
    global playcount
    oldresult = 'none'
    count = 0
    fgbg2 = cv2.createBackgroundSubtractorKNN();
    aWeight = 0.5
    camera = cv2.VideoCapture(0)
    top, right, bottom, left = 10, 350, 325, 690
    num_frames = 0
    while(True):
        (grabbed, frame) = camera.read()
```

```
        frame = imutils.resize(frame, width=700)
        frame = cv2.flip(frame, 1)
        clone = frame.copy()
        (height, width) = frame.shape[:2]
        roi = frame[top:bottom, right:left]
        gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
        gray = cv2.GaussianBlur(gray, (41, 41), 0)
        if num_frames < 30:
            run_avg(gray, aWeight)
        else:
            temp = gray
            hand = segment(gray)
            if hand is not None:
                (thresholded, segmented) = hand
                #cv2.drawContours(clone, [segmented + (right,
top)], -1, (0, 0, 255))
                #cv2.imwrite("test.jpg",temp)
                #cv2.imshow("Thesholded", temp)
                #ret, thresh = cv2.threshold(temp, 150, 255,
cv2.THRESH_BINARY + cv2.THRESH_OTSU)
                #thresh = cv2.resize(thresh, (64, 64))
                #thresh = np.array(thresh)
                #img = np.stack((thresh,)*3, axis=-1)
                roi = frame[top:bottom, right:left]
                roi = fgbg2.apply(roi);
                cv2.imwrite("test.jpg",roi)
                #cv2.imwrite("newDataset/Fist/"+str(count)+".
png",roi)
                #count = count + 1
                #print(count)
                img = cv2.imread("test.jpg")
                img = cv2.resize(img, (64, 64))
                img = img.reshape(1, 64, 64, 3)
                img = np.array(img, dtype='float32')
                img /= 255
                predict = classifier.predict(img)
                value = np.amax(predict)
                cl = np.argmax(predict)
                result = names[np.argmax(predict)]
                if value >= 0.99:
                    translated_text = translate(str(result),'hi
')
                    translated_text1 = translate(str(result),'
te')
                    print(str(value)+" "+str(result)+
translated_text+ translated_text1)
```

```
                    cv2.putText(clone, 'Gesture Recognize as :
    '+str(result), (10, 25),  cv2.FONT_HERSHEY_SIMPLEX,0.5, (0,
    255, 255), 2)
                    if oldresult != result:
                        play(playcount,result)
                        oldresult = result
                        playcount = playcount + 1
                else:
                    cv2.putText(clone, '', (10, 25),  cv2.
    FONT_HERSHEY_SIMPLEX,0.5, (0, 255, 255), 2)
                cv2.imshow("video frame", roi)
        cv2.rectangle(clone, (left, top), (right, bottom),
    (0,255,0), 2)
        num_frames += 1
        cv2.imshow("Video Feed", clone)
        keypress = cv2.waitKey(1) & 0xFF
        if keypress == ord("q"):
            break
     camera.release()
     cv2.destroyAllWindows()



font = ('times', 16, 'bold')
title = Label(main, text='Sign Language Recognition to Text &
    Voice using CNN Advance',anchor=W, justify=CENTER)
title.config(bg='yellow4', fg='white')
title.config(font=font)
title.config(height=3, width=120)
title.place(x=0,y=5)


font1 = ('times', 13, 'bold')
upload = Button(main, text="Upload Hand Gesture Dataset",
    command=uploadDataset)
upload.place(x=50,y=100)
upload.config(font=font1)

pathlabel = Label(main)
pathlabel.config(bg='yellow4', fg='white')
pathlabel.config(font=font1)
pathlabel.place(x=50,y=150)

markovButton = Button(main, text="Train CNN with Gesture Images
    ", command=trainCNN)
```

```
markovButton.place(x=50,y=200)
markovButton.config(font=font1)

predictButton = Button(main, text="Sign Language Recognition
    from Webcam", command=webcamPredict)
predictButton.place(x=50,y=250)
predictButton.config(font=font1)


font1 = ('times', 12, 'bold')
text=Text(main,height=15,width=78)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=450,y=100)
text.config(font=font1)

deleteDirectory()
main.config(bg='magenta3')
main.mainloop()
```

6.2 HandGestureRecognition by Collecting live Data

6.2.1 Collecting Data

```
    import cv2
import numpy as np
import os

# Create the directory structure
if not os.path.exists("/home/kalyangopu/Documents/MajorProject/
    data"):
    os.makedirs("/home/kalyangopu/Documents/MajorProject/data")
    os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train")
    os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test")
    os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/0")
    os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/1")
    os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/2")
```

```
13      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/3")
14      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/4")
15      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/5")
16      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/c")
17      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/f")
18      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/y")
19      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/i")
20      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/l")
21      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/o")
22      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    train/z")
23      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/0")
24      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/1")
25      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/2")
26      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/3")
27      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/4")
28      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/5")
29      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/c")
30      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/f")
31      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/y")
32      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/i")
33      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/l")
34      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/o")
35      os.makedirs("/home/kalyangopu/Documents/MajorProject/data/
    test/z")
```

```
36
37 # Train or test
38 mode = 'test'
39 directory = 'data/'+mode+'/'
40
41 cap = cv2.VideoCapture(0)
42
43 while True:
44     _, frame = cap.read()
45     # Simulating mirror image
46     frame = cv2.flip(frame, 1)
47
48     # Getting count of existing images
49     count = {'zero': len(os.listdir(directory+"/0")),
50              'one': len(os.listdir(directory+"/1")),
51              'two': len(os.listdir(directory+"/2")),
52              'three': len(os.listdir(directory+"/3")),
53              'four': len(os.listdir(directory+"/4")),
54              'five': len(os.listdir(directory+"/5")),
55              'c': len(os.listdir(directory+"/c")),
56              'f': len(os.listdir(directory+"/f")),
57              'i': len(os.listdir(directory+"/i")),
58              'l': len(os.listdir(directory+"/l")),
59              'o': len(os.listdir(directory+"/o")),
60              'y': len(os.listdir(directory+"/y")),
61              'z': len(os.listdir(directory+"/z"))
62              }
63
64     # Printing the count in each set to the screen
65     cv2.putText(frame, "MODE : "+mode, (10, 20), cv2.
    FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
66     cv2.putText(frame, "IMAGE COUNT", (10, 40), cv2.
    FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
67     cv2.putText(frame, "ZERO : "+str(count['zero']), (10, 60),
    cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
68     cv2.putText(frame, "ONE : "+str(count['one']), (10, 80),
    cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
69     cv2.putText(frame, "TWO : "+str(count['two']), (10, 100),
    cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
70     cv2.putText(frame, "THREE : "+str(count['three']), (10,
    120), cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
71     cv2.putText(frame, "FOUR : "+str(count['four']), (10, 140),
     cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
72     cv2.putText(frame, "FIVE : "+str(count['five']), (10, 160),
     cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
```

```
    cv2.putText(frame, "c : "+str(count['c']), (10, 180), cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
    cv2.putText(frame, "f : "+str(count['f']), (10, 200), cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
    cv2.putText(frame, "i : "+str(count['i']), (10, 220), cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
    cv2.putText(frame, "l : "+str(count['l']), (10, 240), cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
    cv2.putText(frame, "o: "+str(count['o']), (10, 260), cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
    cv2.putText(frame, "y : "+str(count['y']), (10, 280), cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)
    cv2.putText(frame, "z : "+str(count['z']), (10, 300), cv2.FONT_HERSHEY_PLAIN, 1, (0,255,255), 1)



    # Coordinates of the ROI
    x1 = int(0.5*frame.shape[1])
    y1 = 10
    x2 = frame.shape[1]-10
    y2 = int(0.5*frame.shape[1])
    # Drawing the ROI
    # The increment/decrement by 1 is to compensate for the bounding box
    cv2.rectangle(frame, (x1-1, y1-1), (x2+1, y2+1), (255,0,0),1)
    # Extracting the ROI
    roi = frame[y1:y2, x1:x2]
    roi = cv2.resize(roi, (64, 64))

    cv2.imshow("Frame", frame)

    #_, mask = cv2.threshold(mask, 200, 255, cv2.THRESH_BINARY)
    #kernel = np.ones((1, 1), np.uint8)
    #img = cv2.dilate(mask, kernel, iterations=1)
    #img = cv2.erode(mask, kernel, iterations=1)
    # do the processing after capturing the image!
    roi = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    _, roi = cv2.threshold(roi, 120, 255, cv2.THRESH_BINARY)
    cv2.imshow("ROI", roi)

    interrupt = cv2.waitKey(10)
    if interrupt & 0xFF == 27: # esc key
        break
```

```
    if interrupt & 0xFF == ord('0'):
        cv2.imwrite(directory+'0/'+str(count['zero'])+'.jpg', roi)
    if interrupt & 0xFF == ord('1'):
        cv2.imwrite(directory+'1/'+str(count['one'])+'.jpg', roi)
    if interrupt & 0xFF == ord('2'):
        cv2.imwrite(directory+'2/'+str(count['two'])+'.jpg', roi)
    if interrupt & 0xFF == ord('3'):
        cv2.imwrite(directory+'3/'+str(count['three'])+'.jpg', roi)
    if interrupt & 0xFF == ord('4'):
        cv2.imwrite(directory+'4/'+str(count['four'])+'.jpg', roi)
    if interrupt & 0xFF == ord('5'):
        cv2.imwrite(directory+'5/'+str(count['five'])+'.jpg', roi)
    if interrupt & 0xFF == ord('c'):
        cv2.imwrite(directory+'c/'+str(count['c'])+'.jpg', roi)
    if interrupt & 0xFF == ord('f'):
        cv2.imwrite(directory+'f/'+str(count['f'])+'.jpg', roi)
    if interrupt & 0xFF == ord('y'):
        cv2.imwrite(directory+'y/'+str(count['y'])+'.jpg', roi)
    if interrupt & 0xFF == ord('i'):
        cv2.imwrite(directory+'i/'+str(count['i'])+'.jpg', roi)
    if interrupt & 0xFF == ord('l'):
        cv2.imwrite(directory+'l/'+str(count['l'])+'.jpg', roi)
    if interrupt & 0xFF == ord('o'):
        cv2.imwrite(directory+'o/'+str(count['o'])+'.jpg', roi)
    if interrupt & 0xFF == ord('z'):
        cv2.imwrite(directory+'z/'+str(count['z'])+'.jpg', roi)



cap.release()
cv2.destroyAllWindows()
"""
d = "old-data/test/0"
newd = "/home/kalyangopu/Documents/MajorProject/data/test/0"
for walk in os.walk(d):
    for file in walk[2]:
        roi = cv2.imread(d+"/"+file)
        roi = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
        _, mask = cv2.threshold(roi, 120, 255, cv2.THRESH_BINARY)
```

```
        cv2.imwrite(newd+"/"+file, mask)
"""
```

6.2.2 Trainig the Data

```
    # Importing the Keras libraries and packages
from keras.models import Sequential
from keras.layers import Convolution2D
from keras.layers import MaxPooling2D
from keras.layers import Flatten
from keras.layers import Dense

# Step 1 - Building the CNN

# Initializing the CNN
classifier = Sequential()

# First convolution layer and pooling
classifier.add(Convolution2D(32, (3, 3), input_shape=(64, 64,
    1), activation='relu'))
classifier.add(MaxPooling2D(pool_size=(2, 2)))
# Second convolution layer and pooling
classifier.add(Convolution2D(32, (3, 3), activation='relu'))
# input_shape is going to be the pooled feature maps from the
    previous convolution layer
classifier.add(MaxPooling2D(pool_size=(2, 2)))

# Flattening the layers
classifier.add(Flatten())

# Adding a fully connected layer
classifier.add(Dense(units=150, activation='relu'))
classifier.add(Dense(units=13, activation='softmax')) # softmax
    for more than 2

# Compiling the CNN
classifier.compile(optimizer='adam', loss='
    categorical_crossentropy', metrics=['accuracy']) #
    categorical_crossentropy for more than 2


# Step 2 - Preparing the train/test data and training the model

# Code copied from - https://keras.io/preprocessing/image/
```

```
from keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
        rescale=1./255,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True)

test_datagen = ImageDataGenerator(rescale=1./255)

training_set = train_datagen.flow_from_directory('C:/Users
    /91938/Documents/handgesNew/data/train',
                                                 target_size
    =(64, 64),
                                                 batch_size=5,
                                                 color_mode='
    grayscale',
                                                 class_mode='
    categorical')

test_set = test_datagen.flow_from_directory('C:/Users/91938/
    Documents/handgesNew/data/test',
                                            target_size=(64,
    64),
                                            batch_size=5,
                                            color_mode='
    grayscale',
                                            class_mode='
    categorical')
classifier.fit_generator(
        training_set,
        steps_per_epoch=25, # No of images in training set
        epochs=100,
        validation_data=test_set,
        validation_steps=40)# No of images in test set


# Saving the model
model_json = classifier.to_json()
with open("model-bw.json", "w") as json_file:
    json_file.write(model_json)
classifier.save_weights('model-bw.h5')
```

6.2.3 Predicting the HandGesture

```
    import numpy as np
from keras.models import model_from_json
import operator
import cv2
import sys, os
import time
import json
from flask import Flask, render_template, Response
from googletrans import Translator

app = Flask(__name__)

# Load the model
model_path = "C:/Users/91938/Documents/handgesNew"  # Assuming
   the model files are in the project directory
model_json_file = os.path.join(model_path, 'model-bw.json')
model_weights_file = os.path.join(model_path, 'model-bw.h5')

with open(model_json_file, 'r') as json_file:
    model_json = json_file.read()

loaded_model = model_from_json(model_json)
loaded_model.load_weights(model_weights_file)
print("Loaded model from disk")

def gen():
    cap = cv2.VideoCapture(0)

    while True:
        _, frame = cap.read()
        frame = cv2.flip(frame, 1)

        x1, y1 = int(0.5 * frame.shape[1]), 10
        x2, y2 = frame.shape[1] - 10, int(0.5 * frame.shape[1])

        cv2.rectangle(frame, (x1-1, y1-1), (x2+1, y2+1), (255,
   0, 0), 1)
        roi = frame[y1:y2, x1:x2]

        roi = cv2.resize(roi, (64, 64))
        roi = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
        _, test_image = cv2.threshold(roi, 120, 255, cv2.
   THRESH_BINARY)

```

```
        result = loaded_model.predict(test_image.reshape(1, 64, 64, 1))
       # print(str(result))
        categories = {0: 'ZERO', 1: 'ONE', 2: 'TWO', 3: 'THREE', 4: 'FOUR', 5: 'FIVE', 7: 'c', 8: 'f', 9: 'i', 10: 'l', 11: 'o', 12: 'y', 6: 'z'}
        prediction = {}
        for i in range(min(len(categories), len(result[0]))):
            if i in categories:
                prediction[categories[i]] = result[0][i]

        prediction = sorted(prediction.items(), key=operator.itemgetter(1), reverse=True)

        # Adding error handling
        if prediction:  # Ensure prediction is not empty
            cv2.putText(frame, prediction[0][0], (10, 120), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2, cv2.LINE_AA)
            def translate(text, target_language):
                translator = Translator()
                translated_text = translator.translate(text, dest=target_language)
                return translated_text.text
            translated_text = translate(prediction[0][0],'hi')
            translated_text1 = translate(prediction[0][0],'te')
            print(prediction[0][0],translated_text, translated_text1)
        else:
            cv2.putText(frame, 'No prediction', (10, 120), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2, cv2.LINE_AA)

        frame_encoded = cv2.imencode('.jpg', frame)[1].tobytes()
        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + frame_encoded + b'\r\n')

    cap.release()
    cv2.destroyAllWindows()

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/video_feed')
def video_feed():
```

```
78      return Response(gen(), mimetype='multipart/x-mixed-replace;
        boundary=frame')
79
80  if __name__ == '__main__':
81      app.run(port=5001)
```

6.3 Face to Emoji Convertion and Predicting the FaceGesture

```
1       import tkinter as tk
2   from tkinter import *
3   import cv2
4   from PIL import Image, ImageTk
5   import os
6   import numpy as np
7   import cv2
8   from keras.models import Sequential
9   from keras.layers import Dense, Dropout, Flatten
10  from keras.layers import Conv2D
11  from keras.optimizers import Adam
12  from keras.layers import MaxPooling2D
13  from keras.preprocessing.image import ImageDataGenerator
14  import threading
15
16
17  emotion_model = Sequential()
18  emotion_model.add(Conv2D(32, kernel_size=(3, 3), activation='
        relu', input_shape=(48,48,1)))
19  emotion_model.add(Conv2D(64, kernel_size=(3, 3), activation='
        relu'))
20  emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
21  emotion_model.add(Dropout(0.25))
22  emotion_model.add(Conv2D(128, kernel_size=(3, 3), activation='
        relu'))
23  emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
24  emotion_model.add(Conv2D(128, kernel_size=(3, 3), activation='
        relu'))
25  emotion_model.add(MaxPooling2D(pool_size=(2, 2)))
26  emotion_model.add(Dropout(0.25))
27  emotion_model.add(Flatten())
28  emotion_model.add(Dense (1024, activation ='relu'))
29  emotion_model.add(Dropout(0.5))
```

```
emotion_model.add(Dense (7, activation='softmax'))
emotion_model.save_weights('model.h5')
cv2.ocl.setUseOpenCL(False)

emotion_dict = {0: " Angry ", 1: " Disgusted", 2: " Fearful",
   3: " Happy ", 4: " Neutral", 5:" Sad",6:"Surprised"}

cur_path = os.path.dirname(os.path.abspath(__file__))

emoji_dist={0:cur_path+"/emojis/angry.png",1:cur_path+"/emojis/
   disgusted.png",2:cur_path+"/emojis/fearful.png",3:cur_path
   +"/emojis/happy.png",4:cur_path+"/emojis/neutral.png",5:
   cur_path+"/emojis/sad.png",6:cur_path+"/emojis/surprised.png
   "}

global last_frame1
last_frame1 = np.zeros((480, 640, 3), dtype=np.uint8)
global capl
show_text=[0]
global frame_number

def show_subject():
    video_path ="C:/Users/91938/Pictures/Camera Roll/
   WIN_20240306_07_43_30_Pro.mp4"
    cap1 = cv2.VideoCapture(video_path)
    if not cap1.isOpened():
        print("Can't open the camera")
    global frame_number
    length = int(cap1.get(cv2.CAP_PROP_FRAME_COUNT))
    frame_number += 1
    if frame_number >= length:
        exit()
    cap1.set(1, frame_number)
    flag1, frame1 = cap1.read()
    frame1 = cv2.resize(frame1, (600,500))
    bounding_box =cv2.CascadeClassifier('C:/Users/91938/
   anaconda3/lib/site-packages/cv2/data/
   haarcascade_frontalface_default.xml')
    gray_frame = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    num_faces=bounding_box.detectMultiScale(gray_frame,
   scaleFactor=1.3, minNeighbors=5)
    for (x, y, w, h) in num_faces:
        cv2.rectangle(frame1, (x, y-50), (x+w, y+h+10), (255,
   0, 0), 2)
        roi_gray_frame = gray_frame[y:y + h, x:x + w]
```

```
65          cropped_img = np.expand_dims(np.expand_dims(cv2.resize(
    roi_gray_frame, (48, 48)), -1), 0)
66          prediction = emotion_model.predict(cropped_img)
67          maxindex = int(np.argmax(prediction))
68          cv2.putText(frame1, emotion_dict[maxindex], (x+20, y
    -60), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2, cv2.
    LINE_AA)
69          show_text[0]=maxindex
70      if flag1 is None:
71          print ("Major error!")
72      elif flag1:
73      # global last_frame1
74          last_frame1=frame1.copy()
75          pic=cv2.cvtColor(last_frame1,cv2.COLOR_BGR2RGB)
76          img=Image.fromarray(pic)
77          imgtk=ImageTk.PhotoImage(image=img)
78          lmain.imgtk=imgtk
79          lmain.configure(image=imgtk)
80          root.update()
81          lmain.after(10,show_subject)
82      if cv2.waitKey(1) & 0xFF ==ord('q'):
83          exit()
84
85  def show_emoji():
86      frame2=cv2.imread(emoji_dist[show_text[0]])
87      pic2=cv2.cvtColor(frame2,cv2.COLOR_BGR2RGB)
88      img2=Image.fromarray(frame2)
89      imgtk2=ImageTk.PhotoImage(image=img2)
90      lmain2.imgtk2=imgtk2
91      lmain3.configure(text=emotion_dict[show_text[0]],font=('
    arial', 45, 'bold'))
92
93      lmain2.configure(image=imgtk2)
94      root.update()
95      lmain2.after(10, show_emoji)
96
97
98  if __name__ == '__main__':
99      frame_number = 0
100     root=tk.Tk()
101     lmain = tk. Label(master=root, padx=50,bd=10)
102     lmain2 = tk.Label(master=root, bd=10)
103     lmain3=tk.Label(master=root, bd=10, fg="#CDCDCD", bg='black
    ')
104     lmain.pack(side=LEFT)
105     lmain.place(x=50,y=250)
```

```
106     lmain3.pack()
107     lmain3.place(x=960,y=250)
108     lmain2.pack(side=RIGHT)
109     lmain2.place(x=900,y=350)
110 
111     root.title("Photo To Emoji")
112     root.geometry("1400x900+100+10")
113     root['bg']='black'
114     exitButton = Button(root, text="Quit", fg="red", command=
        root.destroy, font=('arial', 25, 'bold')).pack(side=BOTTOM)
115 
116 
117     threading.Thread(target=show_subject).start()
118     threading.Thread(target=show_emoji).start()
119     root.mainloop()
```

Chapter 7

TESTING

7.1 Testing of HandGestures Recognition

7.1.1 White Box Testing

White box testing involves examining the internal logic and structures of the code, such as the background subtraction algorithm and CNN model training process. Testers analyze control flow paths, variable scopes, and error handling mechanisms to ensure correctness, efficiency, and adherence to coding standards.

7.1.2 Black Box Testing

Black box testing focuses on assessing the external behavior and functionalities of the program, including dataset uploading, webcam gesture recognition, and voice output. Testers evaluate input-output relationships, user interfaces, and system interactions to verify correct responses and compliance with functional requirements.

7.1.3 Unit Testing

Unit testing entails testing individual components and functions, like background subtraction and CNN model training, in isolation. Testers validate inputs, outputs, and behaviors through test cases and assertions, ensuring code reliability, maintainability, and adherence to specifications.

7.1.4 Integration Testing

Integration testing evaluates interactions between different modules, such as dataset handling, CNN model training, and webcam gesture recognition, ensuring seamless communication and functionality integration. Testers verify data flows, message passing, and error handling scenarios to ensure system robustness and reliability under various operating conditions.

7.1.5 Validation Testing

Validation testing validates the program against user requirements, expectations, and acceptance criteria, ensuring it meets stakeholders' needs and delivers the intended value. Testers assess the accuracy of hand gesture recognition, voice output, and usability aspects through acceptance testing with end-users or domain experts.

7.1.6 System Testing

System testing validates the entire system's behavior, including dataset handling, CNN model training, webcam gesture recognition, and voice output, ensuring compliance with functional requirements and specifications. Testers evaluate performance, scalability, and reliability aspects to ensure a seamless user experience across different environments and scenarios.

7.2 Testing of HandGestureRecognition for live Data Collection

7.2.1 White Box Testing

Internally examines gesture recognition system, validating model loading, preprocessing, and prediction mechanisms. Tests code paths, input-output relationships, and error handling for robustness.

7.2.2 Black Box Testing

Externally evaluates gesture recognition functionality, focusing on webcam inputs and UI responsiveness. Verifies accurate gesture interpretation without internal details consideration.

7.2.3 Unit Testing

Targets individual components, like model loading and preprocessing, verifying functionality in isolation. Validates handling of edge cases and expected outputs for robustness.

7.2.4 Integration Testing

Evaluates module interaction, ensuring seamless communication and data flow between components. Validates interoperability and accuracy in producing timely results.

7.2.5 Validation Testing

Validates system functionality against predefined requirements and user expectations. Tests gesture recognition accuracy, prediction delivery, and interface responsiveness.

7.3 Testing of Face to Emoji Convertion and Prediction the FaceGesture

7.3.1 System Testing

Verifies complete system behavior, including frontend, backend, and external component integration. Ensures performance, reliability, and scalability under real-world conditions.

7.3.2 White Box Testing

This method delves into the internal workings of the code, ensuring that the CNN model and face detection algorithms function correctly and efficiently. Testers meticulously examine control flow paths, variable scopes, and error handling mechanisms to pinpoint potential issues and enhance the overall quality of the code.

7.3.3 Black Box Testing

Here, the focus shifts to the external behavior of the program, particularly its handling of tasks like video input processing and emotion recognition. Testers assess how the system responds to various inputs and scenarios, validating its functionality and usability without needing insight into its internal implementation.

7.3.4 Unit Testing

In this phase, individual components and functions, such as the emotion recognition model and face detection algorithm, are rigorously tested in isolation. Testers meticulously validate inputs, outputs, and behaviors against predefined test cases, ensuring the reliability and accuracy of these components while improving code quality through early bug detection and refinement.

7.3.5 Integration Testing

This phase evaluates how different modules like emotion recognition and video processing interact, ensuring smooth communication and integration of functionalities. Testers validate data flows, message passing, and error handling to assess end-to-end functionality and ensure system interoperability across various scenarios.

7.3.6 Validation Testing

Here, the program undergoes validation against user requirements and expectations to ensure it fulfills stakeholders' needs. Testers scrutinize emotion recognition accuracy, avatar display correctness, and user interface elements through acceptance testing, aligning the program with project objectives and delivering a satisfactory user experience.

7.3.7 System Testing

This testing phase assesses the overall behavior of the entire system, including emotion recognition and video processing functionalities. Testers evaluate performance, scalability, and reliability by simulating real-world usage scenarios, verifying seamless operation and user satisfaction across diverse environments and use cases.

Chapter 8

CONCLUSION

Gesture Recognition Technology interpret human gestures and actions via mathematical algorithms. This technology is very useful in robotic and gaming zone and makes the control more easier. Every God creature has an importance in the society, remembering this fact, let us try to include hearing impaired people in our day to day life and live together.It helps and aids dumb and deaf people to live independently. It eliminates gap among people and leads to achieve better society.

REFERENCES

[1] *Gesture Recognition System — IEEE Conference Publication.*

[2] *Hand Gesture Recognition for Human Computer Interaction*

[3] *Long Hands gesture recognition system: 2 step gesture recognition with machine learning and geometric shape analysis — Multimedia Tools and Applications*

[4] *The International Gesture Recognition Association (IGRA)*

[5] *Kaggle Datasets.*